



深圳大学
Shenzhen University

互联网编程

第2讲 网络数据流

(java网络程序的输入输出)

深圳大学计算机与软件学院

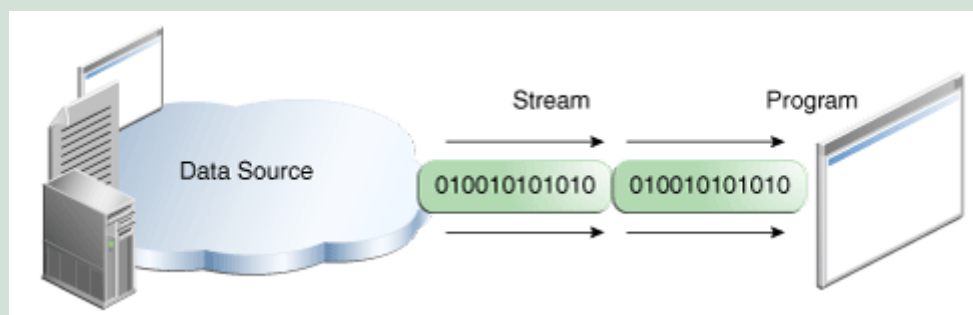
主要内容

- 网络程序的输入输出
- **Java I/O流分析**
- 缓冲字符流编程
- 文件字节流编程
- 数据流编程
- 对象流编程
- 字节/字符数组流编程
- 网络数据流编码与安全读写问题

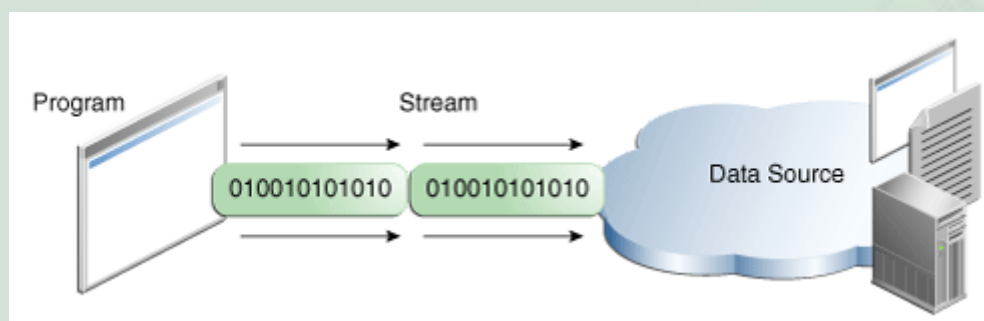


网络程序的输入输出

- 程序大都要完成读写数据工作
- 将数据从一个系统移动到另一个系统



Reading information into a program.



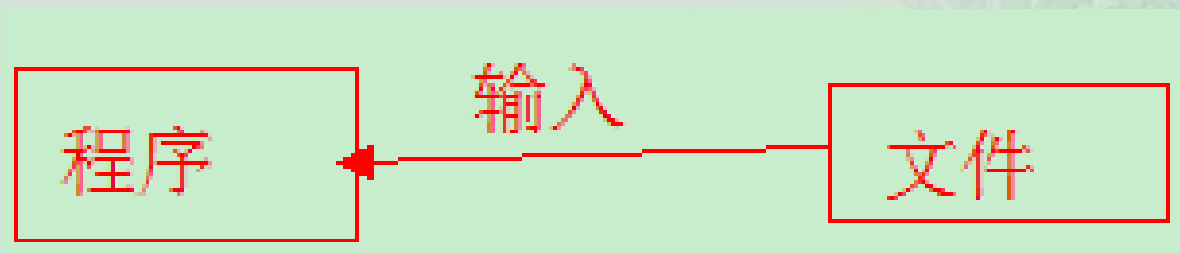
Writing information from a program.



transmitting information between programs.

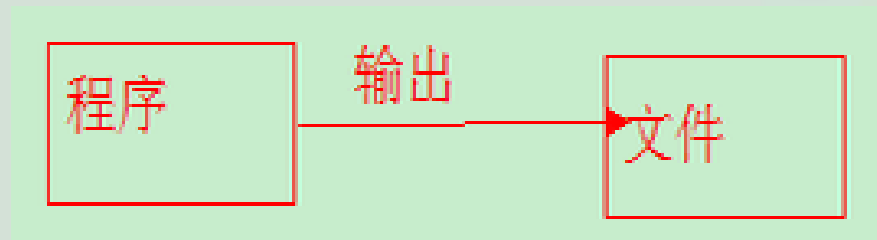
JAVA I/O 流分析

- Java程序读写文件时可以使用输入/输出流，简称I/O流。
- 输入和输出都是从程序的角度来说。
- 输入流（input stream or input object）的数据来源称作“源”
- 程序从输入流中读取“源”中的数据



JAVA I/O 流分析

- **输出流**（output stream or output object）的指向称作“**目的地**”
- 程序通过向**输出流**中写入数据，把信息传递到“**目的地**”



- 程序的“源”和“目的地”可以是文件、键盘、鼠标、内存或显示器窗口、网络

JAVA I/O 流分析

- 节点流：直接与数据源相连，读入或读出。



读写字节数据的节点流：

FileInputStream/FileOutputStream

PipedInputStream/PipedOutputStream

ByteArrayInputStream/ByteArrayOutputStream

读写字符数据的节点流：

FileReader/FileWriter

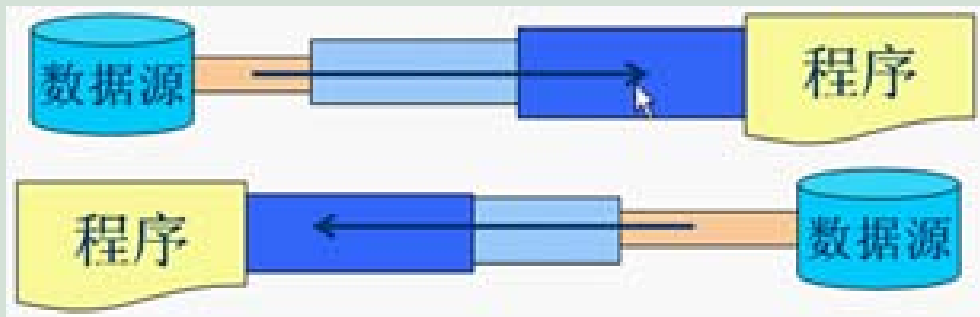
PipedReader/PipedWriter

CharArrayReader/CharArrayWriter



JAVA I/O 流分析

- 直接使用节点流，读写不方便，为了更快的读写文件，才有了处理流。
- 处理流：与节点流一块使用，在节点流的基础上，再套接一层，套接在节点流上的就是处理流



快速读写字节数据的处理流：

BufferedInputStream/BufferedOutputStream

SequenceInputStream/SequenceOutputStream

DataInputStream/DataOutputStream

ObjectInputStream/ObjectOutputStream

快速读写字符数据的处理流：

BufferedReader/BufferedWriter

InputStreamReader/OutputStreamWriter

PrintReader/PrintWriter

Byte



JAVA I/O流分析

➤ 根据数据传输特性将流抽象为各种类，方便更直观地进行数据操作。

➤ java.io中有4个重要的abstract class:

1. InputStream (字节输入流)

2. OutputStream (字节输出流)

3. Reader (字符输入流)

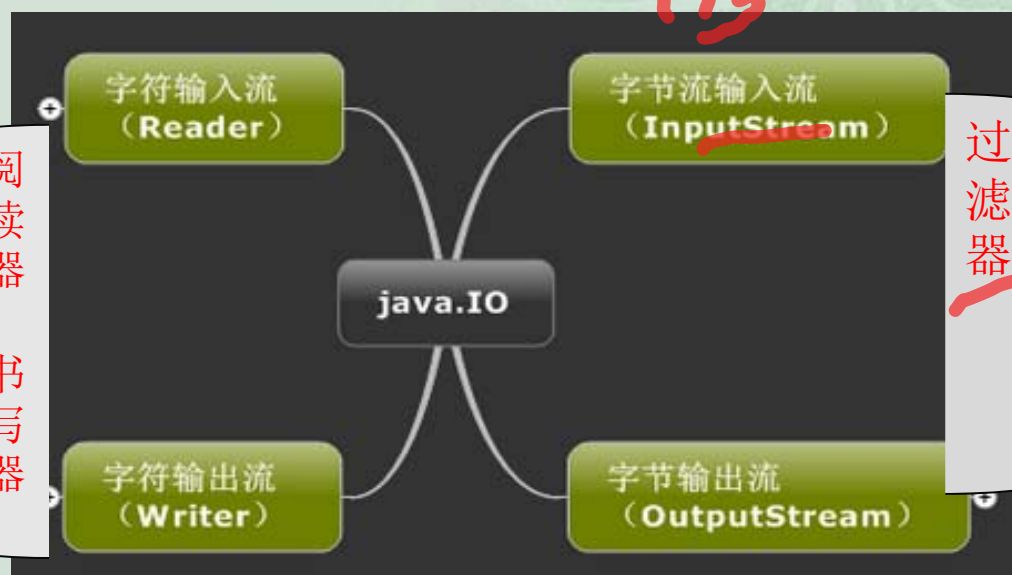
4. Writer (字符输出流)

一次读入或输出8位二进制

一次读入或输出16位二进制

阅读器
书写器

过滤器



JAVA I/O 流分析

➤ 对管道进行操作：PipedInputStream的一个实例要和PipedOutputStream的一个实例共同使用，共同完成管道的读取写入操作。主要用于线程操作。

➤ 字节/字符数组操作：是在内存中开辟了一个字节或字符数组。

- ❑ ByteArrayInputStream
- ❑ ByteArrayOutputStream
- ❑ CharArrayReader
- ❑ CharArrayWriter



JAVA I/O 流分析

➤ **Buffered缓冲流**：是带缓冲区的处理流，缓冲区的作用是避免每次和硬盘或网络打交道，提高数据访问或传输的效率。

- ❑ BufferedInputStream
- ❑ BufferedOutputStream
- ❑ BufferedReader
- ❑ BufferedWriter

缓冲流



JAVA I/O 流分析

➤ 转化流：把字节转化成字符。

□ InputStreamReader

□ OutputStreamWriter

➤ 若是我们输出一个8个字节的long类型或4个字节的float类型，那怎么办呢？

➤ 可以一个字节一个字节输出，也可以转换成字符串输出，但是这样转换费时间，若能直接输出该多好！

★ 数据流：可以直接输出float类型或long类型，提高了数据读写的效率。

□ DataInputStream

□ DataOutputStream



JAVA I/O 流分析

➤ 打印流：是打印输出到控制台。

□ printStream

□ printWriter

➤ 对象流：把封装的对象直接输出，而不是一个个在转换成字符串再输出。

□ ObjectInputStream

□ ObjectOutputStream

➤ 使用对象流需要实现Serializable接口，否则会报错。

➤ 序列化流：把对象直接转换成二进制，写入介质中。

□ SequenceInputStream

K-V 输出



例析-- 缓冲字符流

- 1. `BufferedReader`类
- `BufferedReader`的构造方法：
 - ☞ `BufferedReader(Reader in)`
 - ☞ `BufferedReader`流能够读取文本行，方法是 `readLine()`

文本数据的读取和分析中用的比较多的是 `FileReader` 和 `BufferedReader`，
达到按行读取的目的

例析--- 缓冲字符流

- 通过向BufferedReader传递一个Reader对象（如FileReader的对象），来创建一个BufferedReader对象，如：

```
FileReader fr = new FileReader("Student.txt");  
BufferedReader input = new BufferedReader(fr);
```

- 然后，input调用readLine()顺序读取文件Student.txt的一行。

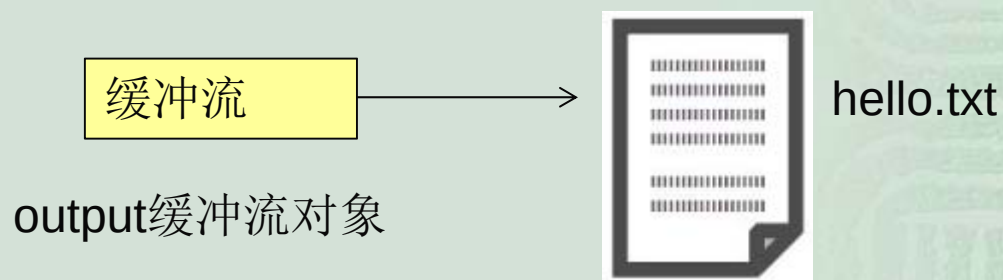


例析-- 缓冲字符流

- 2.BufferedWriter类
- 类似地，可以将BufferedWriter流和FileWriter流连接在一起，然后使用BufferedWriter流将数据写到目的地，例如：

```
FileWriter fw = new FileWriter("hello.txt");  
BufferedWriter output = new BufferedWriter(fw);
```

- BufferedWriter流调用如下方法，把字符串s或s的一部分写入到目的地
 - ❧ write(String s)
 - ❧ write(String s, int off, int len)

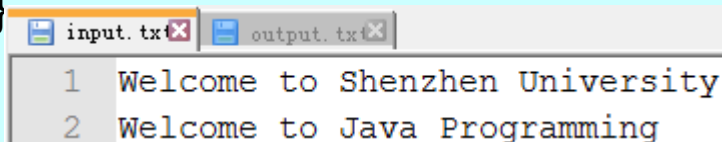


```

import java.io.*;
public class Example9_5{
    public static void main(String args[]){
        try{
            < FileReader fr = new FileReader("input.txt");
            < BufferedReader input = new BufferedReader(fr);
            < FileWriter fw = new FileWriter("output.txt");
            < BufferedWriter output = new BufferedWriter(fw);

            String s=null;
            int i=0;
            while((s = input.readLine())!=null){
                i++;
                output.write(i + ": " + s);
                output.newLine();
            }
            output.flush(); output.close(); input.close();
            fw.close(); fr.close();
        }
        catch(IOException e){
            System.out.println(e);
        }
    }
}

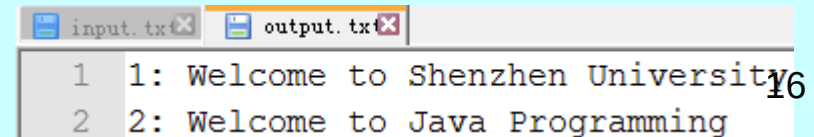
```



```

input.txt
1 Welcome to Shenzhen University
2 Welcome to Java Programming

```



```

output.txt
1 1: Welcome to Shenzhen University
2 2: Welcome to Java Programming

```

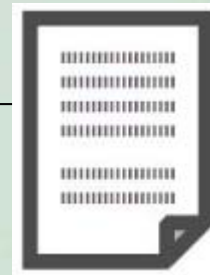
按字节读写数据--文件字节流

- 1. `FileInputStream`类
- 为了创建`FileInputStream`类的对象，可以使用下列构造方法：
 - ❧ `FileInputStream(String name)`
 - ❧ `FileInputStream(File file)`



字节流

以字节为
单位读



例析--文件字节流

- 2.FileOutputStream类

- 构造方法

- ❧ FileOutputStream(String name)

- ❧ FileOutputStream(File file)

- 输出流通过使用write()方法把数据写入输出流到达目的地

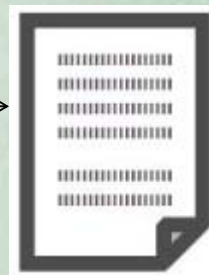
- ❧ public void write(byte b[]): 写**b.length**个**字节**到输出流

- ❧ public void write(byte b[],int off,int len): 从给定字节数组中起始于偏移量**off**处写**len**个字节到输出流，参数**b**是存放了数据的字节数组

字节流



以字节为
单位写



```
import java.io.*;
public class Example9_3
```

思考：这段程序对应的数据读写场景是怎样的？

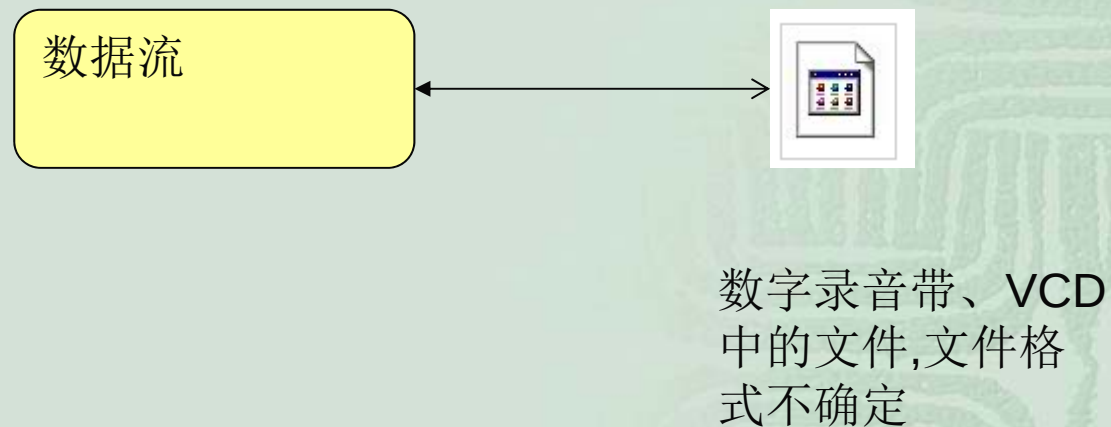
```
{
    public static void main(String args[])
    {
        File file = new File("hello.txt");
        byte b[] = "深圳大学".getBytes();
        try{
            FileOutputStream output = new FileOutputStream(file);
            output.write(b); // 字节数组
            output.close();

            FileInputStream input = new FileInputStream(file);
            int n=0;
            while( (n=input.read(b,0,2))!=-1 ) // 最多读2个字节
            {
                String str = new String(b,0,n); // 转换为字符串
                System.out.println(str);
            }
        }
        catch(IOException e){
            System.out.println(e);
        }
    }
}
```

深圳大学

数据流

- 1.DataInputStream类和DataOutputStream类
- DataInputStream类创建的对象称为数据输入流
- DataOutputStream类创建的对象称为数据输出流



```
import java.io.*;
public class Example9_8
```

思考：这段程序对应的数据读写场景是怎样的？

```
{
    public static void main(String args[])
    {
        try{
            FileOutputStream fos = new FileOutputStream("jerry.dat");
            DataOutputStream output = new DataOutputStream(fos);
            output.writeInt(100);
            output.writeChars("I am ok");
        }
        catch(IOException e){}

        try{
            FileInputStream fis = new FileInputStream("jerry.dat");
            DataInputStream input = new DataInputStream(fis);
            System.out.println(input.readInt());
            char c;
            while((c=input.readChar())!='\0') //' \0'表示空字符
                System.out.print(c);
        }
        catch(IOException e){}
    }
}
```

```
100
I am ok
```

对象流

- 1. `ObjectInputStream`类和`ObjectOutputStream`类
- `ObjectInputStream`类创建的对象被称为对象输入流
- `ObjectOutputStream`类创建的对象被称为对象输出流
- 对象输出流使用`writeObject(Object obj)`方法将一个对象obj写入输出流
- 对象输入流使用`readObject()`从源中读取一个对象到程序中
- 一个类如果实现了`Serializable`接口，那么这个类创建的对象就是序列化的对象（a serializable object）。
- 当把一个序列化的对象写入到对象输出流时，JVM会自动按着一定格式的文本将对象写入到目的地。

```
import java.io.*;
class Goods implements Serializable
{
    String name = null;
    double unitPrice;
    Goods(String name, double unitPrice){
        this.name=name;
        this.unitPrice=unitPrice;
    }
    public void setUnitPrice(double unitPrice){
        this.unitPrice=unitPrice;
    }
    public double getUnitPrice(){
        return unitPrice;
    }
    public void setName(String name){
        this.name=name;
    }
    public String getName(){
        return name;
    }
}
```



思考：这段程序对应的数据读写场景是怎样的？

例析--对象流

```
public class Example9_9
{
    public static void main(String args[])
    {
        Goods TV1 = new Goods("HaierTV",3468);
        try{
            FileOutputStream fileOut = new FileOutputStream("a.txt");
            ObjectOutputStream objectOut = new ObjectOutputStream(fileOut);
            → objectOut.writeObject(TV1);

            FileInputStream fileIn = new FileInputStream("a.txt");
            ObjectInputStream objectIn = new ObjectInputStream(fileIn);
            → Goods TV2 = (Goods)objectIn.readObject();

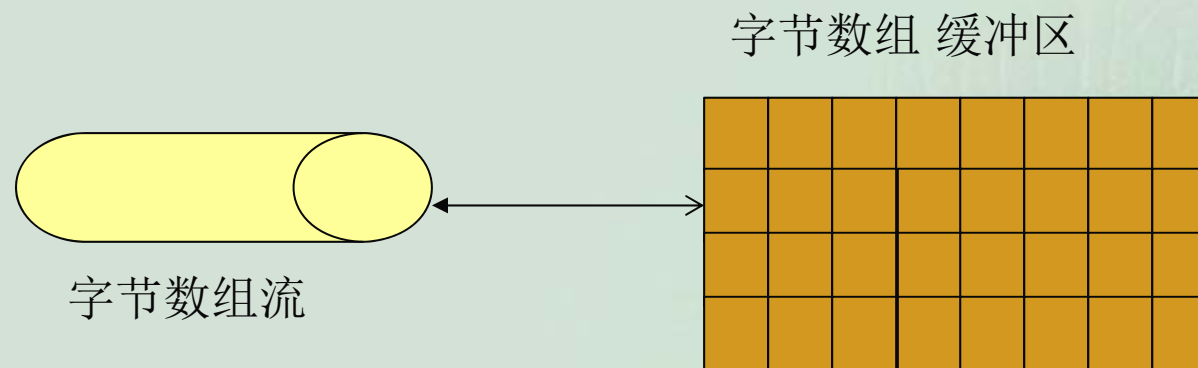
            TV2.setUnitPrice(8888);
            TV2.setName("GreatWall");
            System.out.printf("\nTv1:%s,%f",TV1.getName(),TV1.getUnitPrice());
            System.out.printf("\nTv2:%s,%f",TV2.getName(),TV2.getUnitPrice());
        }
        catch(Exception event){
            System.out.println(event);
        }
    }
}
```

```
Tv1:HaierTV,3468.000000
Tv2:GreatWall,8888.000000
```

例析— 字节数组流

使用字节数组作为流的源和目的地

- 字节输入流: `ByteArrayInputStream`
- 字节输出流: `ByteArrayOutputStream`
- 分别使用**字节数组**作为流的源和目标



例析—字节数组流

返回输出流写入到缓冲区的全部字节

```
import java.io.*;
public class Example9_6
{
    public static void main(String args[])
    {
        int n=-1;
        ByteArrayOutputStream output = new ByteArrayOutputStream();
        for(int i=0;i<5;i++)
        {
            output.write('A'+i);
        }

        ByteArrayInputStream input = new ByteArrayInputStream(output.toByteArray());
        while((n=input.read())!=-1)
        {
            System.out.println(n + ":" + (char)n);
        }
    }
}
```

65:A
66:B
67:C
68:D
69:E

- 读写场景：
- 向**内存**（输出流的缓冲区）写入ASCII表，然后再读出这些字节和字节对应的字符。

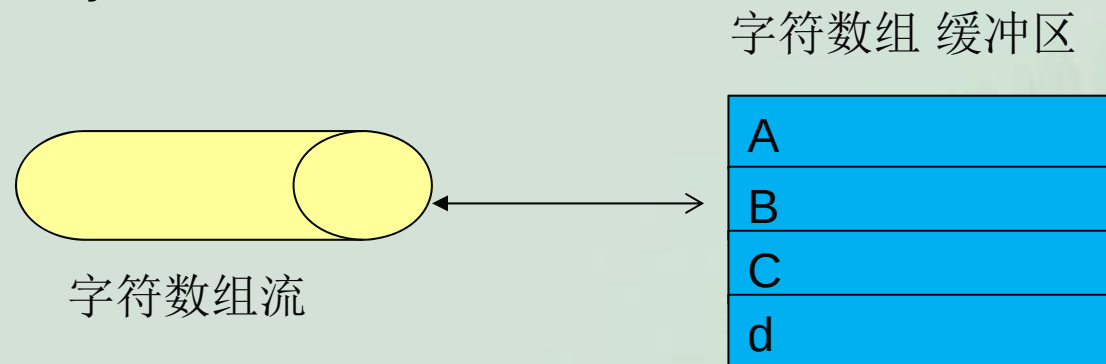
例析—字符 数组流

使用字符数组作为流的源和目的地

- 与数组**字节**流对应的是数组**字符**流

☞ CharArrayReader

☞ CharArrayWriter



```
import java.io.*;
public class Example9_7
```

返回输出流写入到缓冲区的全部字符

```
{
    public static void main(String args[])
    {
        int n=-1;
        CharArrayWriter output = new CharArrayWriter();
        for(int i=65;i<=69;i++)
        {
            output.write(i);
        }

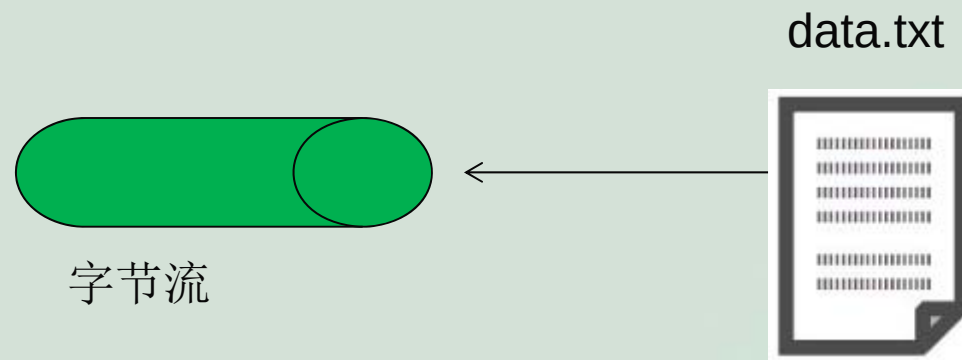
        CharArrayReader input = new CharArrayReader(output.toCharArray());
        try
        {
            while((n=input.read())!=-1)
            {
                System.out.println(n + ":" + (char)n);
            }
        }
        catch(IOException e){}
    }
}
```

65:A
66:B
67:C
68:D
69:E

- 读写场景：
- 将Unicode表中的一些字符写入内存，然后再读出。

应用场景问题：

- 将data.txt文件中保存的信息缓冲输入到程序中



编程时：如何创建读入流？



应用场景问题：（串链过滤器编程）

代码方式1:

```
FileInputSteam fin = new FileInputStream("data.txt");  
BufferedInputStream bin = new BufferedInputStream(fin);  
可以使用fin, bin 访问data.txt
```

代码方式2:

```
InputStream in = new FileInputStream("data.txt");  
in = new BufferedInputStream(in);  
只能使用 in 访问data.txt
```

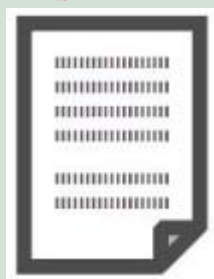
代码方式3: 过滤器与数据流绑定，不能断开

```
DataOutputStream dout = new DataOutputStream(  
    new BufferedOutputStream(  
        new FileOutputStream("data.txt")  
    )  
);
```

应用场景问题：数据格式不一致，需编码转换

- 将JSP或Servlet中的请求数据从UTF8转换为GB2312

stu.jsp/ tea.servlet



http请求

Http协议

server

编程时：如果直接从http流中提取中文数据，提取的结果通常是“?????”，因此需要转换成可理解的中文字符，把UTF8转换成GB2312。如何编程实现转换？



应用场景问题：编码格式转换编程

```
byte [] b;  
String utf8_value;
```

```
/*从HTTP流中取"NAME"的UTF8数据*/
```

```
utf8_value = request.getParameter("NAME");
```

```
① b = utf8_value.getBytes("8859_1"); //中间用ISO-8859-1过渡
```

```
② String name = new String(b, "GB2312"); //转换成GB2312字符
```

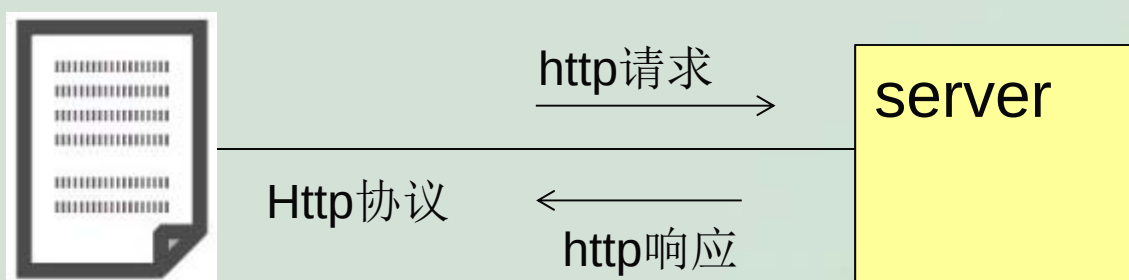
兼容转换

指定格式

应用场景问题：网络数据流的读取

- 在程序中直接将读入的网络数据流转换为等长字节数组？存在安全问题，因在网络获取数据不流畅，数据流的传送会断断续续，不能保证一次能读取全部数据。特别是在读取大容量网络数据时问题更严重。

stu.jsp/ tea.servlet



如何编程安全读取网络数据流？



应用场景问题：安全读取网络数据流编程

在读取数据时检测实际读到的长度，如果没有读完已知长度的数据就应该再次读取，以此循环检测，直到实际读取的长度累加与已知的长度相等。

```
ServletInputStream inStream = request.getInputStream(); //取HTTP请求流
int size = request.getContentLength(); //取HTTP请求流长度
byte[] buffer = new byte[size]; //用于缓存每次读取的数据
byte[] in_b = new byte[size]; //用于存放结果的数组
int count = 0;
int rbyte = 0;
while (count < size) { //循环读取
    rbyte = inStream.read(buffer); //每次实际读取长度存于rbyte中
    for(int i=0;i<rbyte;i++){
        in_b[count + i] = buffer[i];
    }
    count += rbyte;
}
```



练习作业

将你的姓名、学号、班级信息、所用电脑名称按行保存到一个名为client.txt文件中。修改上一章PPT中socket编程例子程序example11-5，从client端逐行读取client.txt文件，每读取一行信息，客户端将这行信息发送到server端，server端在这一行信息最前面加上当前时间信息（如2024年02月26日08: 26: 51），发回client，client端收到后，将收到的信息作为txt文件的一行信息保存到client端一个名为serverResponse.txt文件中。

Client.txt文件中文本信息如下：

姓名：张一山

学号：20210058

班级：计算机科学与技术02班

电脑名：dellZYS



课后拓展思考学习

- 1.网络数据流的读写，在编程应用时经常会遇到哪些问题？
- 2.针对1中的问题，编程时如何应对？

